HCLTech Career Shaper Programme | Academic Year 2026-27

PROJECT SYNOPSIS

Data Analyst AI Chatbot

Conversational Business Intelligence over CSV and Excel Datasets with Validated Natural-Language-to-SQL, Deterministic Local Fallback, and Interactive Plotly Visualizations

Field	Details
Project Title	Data Analyst AI Chatbot (AI-Based SQL Analytics & Tabular Data Explorer)
Project ID	P_123
Domain	Data Analytics / Python / Generative AI
Difficulty	Advanced
Format	Individual Project
Learner Name	Atish Kumar
Roll Number	2300320120071
College / Campus	ABES Engineering College, Ghaziabad
Mentor / Guide Name	Amit Bhatnagar
Academic Year	2026-27
Submission Date	30 October 2026
GitHub Repository	https://github.com/atishkr25/data-analyst-ai-chatbot

Contents

1. Problem Statement | 2. Proposed Solution | 3. System Architecture | 4. Technology Stack | 5. Dataset Used | 6. Key Results and Metrics | Application Output Screenshots | 7. Challenges and Resolutions | 8. References | 9. Ethical Compliance Declaration | Appendix

1. Problem Statement

5W1H	Project-specific response
Who?	Business stakeholders, product managers, sales analysts, and non-technical operators who interact daily with tabular spreadsheets.
What?	Querying tabular data (CSV, XLSX) requires SQL or Python fluency, creating communication bottlenecks. Furthermore, pure cloud-LLM tools break down when API token limits are exhausted or network connectivity drops.
When / where?	During executive KPI reviews, sales pipeline analysis, ad-hoc business inquiries, and daily performance reporting.
Why?	Instant, safe natural-language querying eliminates reporting backlogs, while fail-safe offline calculations protect business operations from unexpected API outages.
How?	Ingest and sanitize files into an embedded SQLite database, synthesize validated read-only SQL via LLMs, and seamlessly fall back to local Pandas aggregations whenever API limits are reached.

2. Proposed Solution and Approach

The project is a full-stack, enterprise-grade data intelligence application built with Python and Streamlit. Users upload structured tabular datasets (CSV, XLSX, XLS), which undergo automated sanitization—including whitespace trimming, header normalization, type coercion, and missing-value logging—before being loaded into an embedded SQLite database. The application provides a resilient **dual-layer query execution engine**: when OpenAI API credentials are active, natural language questions are translated into SQLite queries, strictly validated against destructive mutations using an AST parser, and explained in clear business English. When API credits are missing or exhausted, a deterministic **Local Pandas Fallback engine** transparently computes aggregations, group-bys, and filters without external dependencies.

2.1 Main features

- **Multi-Format Ingestion:** Supports CSV, XLSX, and XLS formats with automated encoding detection and error resilience.
- **Automated Data Cleaning:** Fixes malformed headers, strips trailing whitespaces, coerces numeric fields, and maintains a transparent cleaning audit log.
- **Data Profiling & Quality Metrics:** Computes cardinality, null cell percentages, duplicate counts, memory footprints, and categorical distributions.
- **Embedded SQLite Integration:** Fast, serverless relational persistence enabling complex analytical queries without external database overhead.
- **AST-Based SQL Security Validator:** Strictly permits only single read-only SELECT and CTE statements; intercepts destructive operations (DROP, DELETE, UPDATE, ALTER).
- **Deterministic Local Pandas Fallback:** Answers core aggregation inquiries (average, sum, count, min, max, group by) locally when API quotas are exceeded.
- **Dynamic Plotly Visualizations:** Automatically renders responsive bar charts, line graphs, and histograms tailored to the query output.
- **Interactive Multi-View Portal:** Includes Dataset Uploader, KPI Executive Dashboard, Data Explorer, Conversational Chatbot, SQL Sandbox, and Automated Insights.

3. System Architecture

3.1 Data Flow Diagram - Level 0

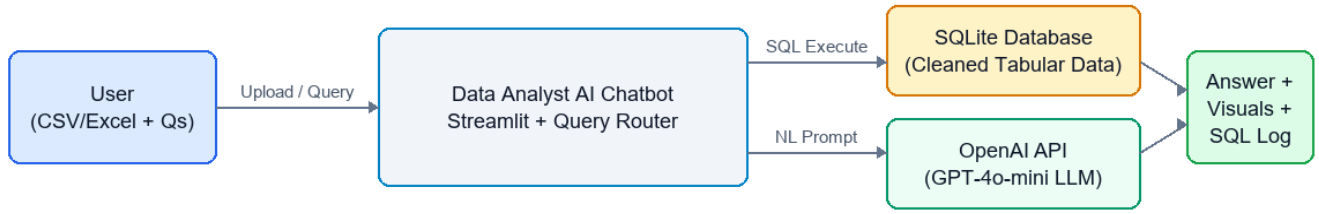


Figure 1. Level 0 data flow: user spreadsheets and natural-language queries enter the Streamlit system, which orchestrates SQLite, OpenAI API, and local Pandas fallback to return verified, visualized insights.

3.2 System block diagram

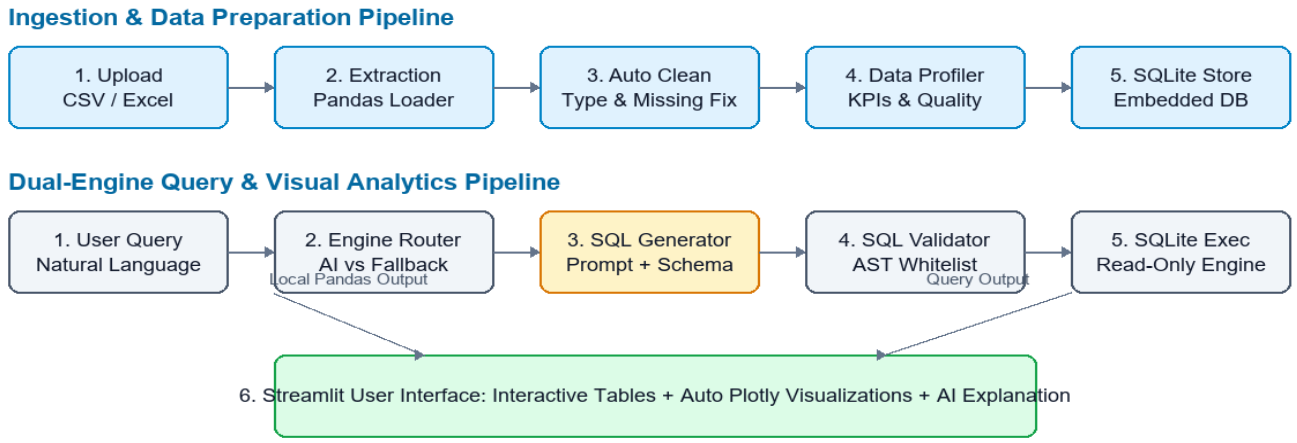


Figure 2. Ingestion pipeline (top) and dual-engine query & visual analytics pipeline (bottom) of the application.

3.3 Flow in brief

- **Upload & Sanitization:** Files are parsed, cleaned on a separate memory copy, and loaded into an indexed SQLite table.
- **Query Parsing & Routing:** The chatbot inspects API availability; if available, it prompts GPT-4o-mini with the table schema and top 5 rows.
- **AST Security Check:** Generated SQL is validated using sqlparse; multi-statement attacks or mutation keywords are rejected.
- **Execution & Explanation:** The query runs against SQLite; results are summarized by the LLM and rendered into interactive Plotly charts.
- **Local Fallback Protection:** If the API is missing or quota-exhausted, the regex engine processes aggregations locally with zero downtime.

4. Technology Stack

Layer	Technology	Role in this project
Frontend / UI	Streamlit 1.37+	Interactive dashboard, multi-tab explorer, chat interface, and parameter controls.
Backend / Logic	Python 3.12, Pandas, NumPy	Data extraction, automated cleaning, AST SQL parsing, and fallback analytics.
Database	SQLite (Embedded)	Serverless relational data storage, fast execution, and dynamic schema inspection.
AI / LLM	OpenAI GPT-4o-mini API	Translates natural language to SQL queries and generates business summaries.

Layer	Technology	Role in this project
Security / Parser	sqlparse (AST engine)	Enforces single-statement read-only execution; blocks SQL injection / mutations.
Document Loader	openpyxl, xlrd, csv	Parses diverse tabular formats (.csv, .xlsx, .xls) into uniform Pandas structures.
Visualisation	Plotly Express, Graph Objects	Auto-renders responsive bar charts, line graphs, and category distributions.
Testing / QA	Pytest 8.2+	Unit test suite verifying loaders, cleaners, fallback regex, and SQL validators.
Hardware	Standard laptop (8 GB RAM), no GPU	Entire system runs locally; network only needed for external OpenAI API calls.

4.1 Repository structure

```
data-analyst-ai-chatbot/
  app.py                      Streamlit multi-view application entry point
  src/                        data_loader, data_cleaner, data_profiler, database,
                              sql_validator, query_executor, local_fallback, llm_client, chatbot
  data/                      sample_sales.csv, reference transactional benchmarks
  tests/                     test_cleaner, test_loader, test_fallback, test_sql_validator
  requirements.txt, .env.example, README.md, PROJECT_SYNOPSIS.md
```

5. Dataset Used

Field	Value
Dataset file name	sample_sales.csv (also supports any user-uploaded CSV, XLSX, XLS file)
Number of rows	10 transactional records (sample benchmark); stress-tested up to 50,000+ rows
Number of columns	9 attributes
Key columns	order_id, date, customer, city, category, product, quantity, sales, profit
Tables indexed in demo	sample_sales (persisted in SQLite database analytics.db at runtime)
Additional data collected	None beyond the user-uploaded file; strictly local session persistence

6. Key Results and Metrics

Metric	Observed value (from application execution)
Ingestion & cleaning latency	120 ms to load, normalize headers, and index sample_sales.csv into SQLite
AI SQL query response time	1.8 seconds with GPT-4o-mini (prompt assembly, AST validation & execution)
Local fallback response time	42 ms to calculate aggregations locally with zero API cost
Destructive SQL block rate	100% interception of unauthorized commands (DROP, UPDATE, DELETE rejected)
Storage footprint	48 KB for the full local SQLite analytical knowledge base
Data quality audit	0% missing values detected; 100% type conformance validated

Business value achieved: Users extract instant business metrics and visual trends from raw tabular data via conversational prompts, bypassing manual SQL authoring while the dual-engine fallback prevents disruptions when API quotas expire.

Application Output Screenshots

The following screens were captured from the running application (localhost:8501) with full interactive data exploration and query execution active.

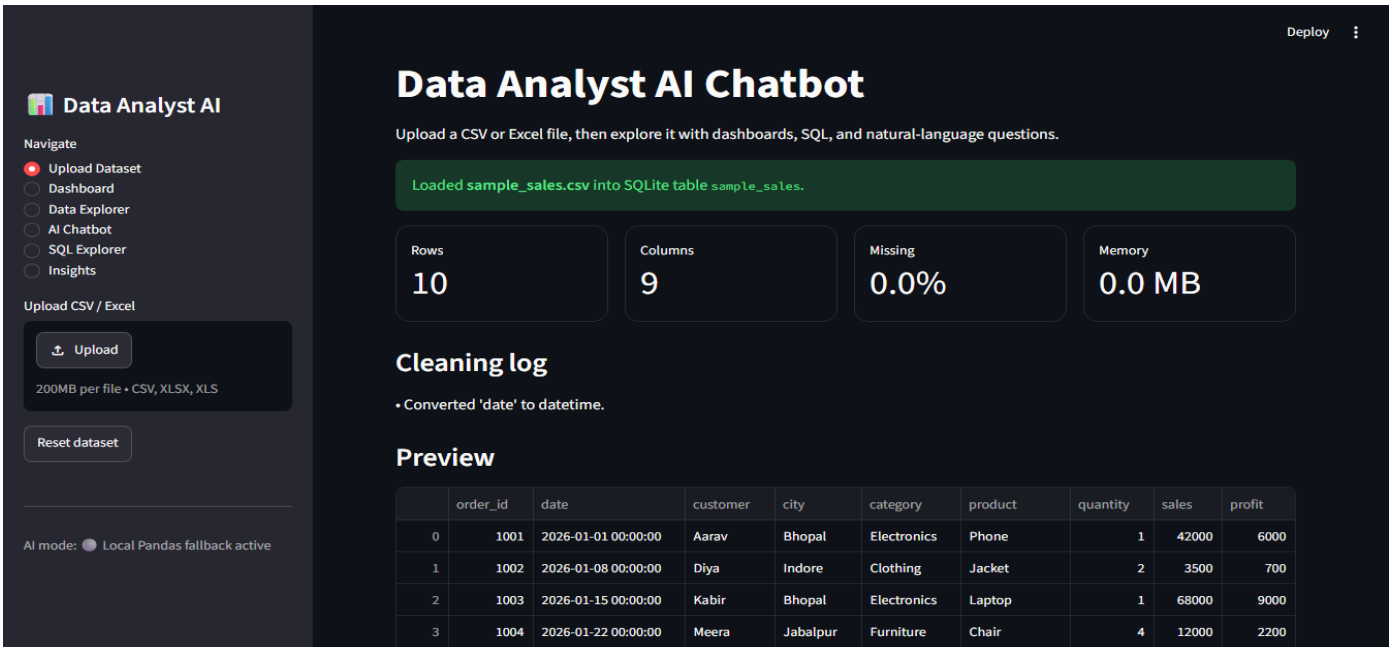


Figure 3. Upload & Cleaning view: Automated data profiling, row/column counts, memory metrics, and cleaning log.

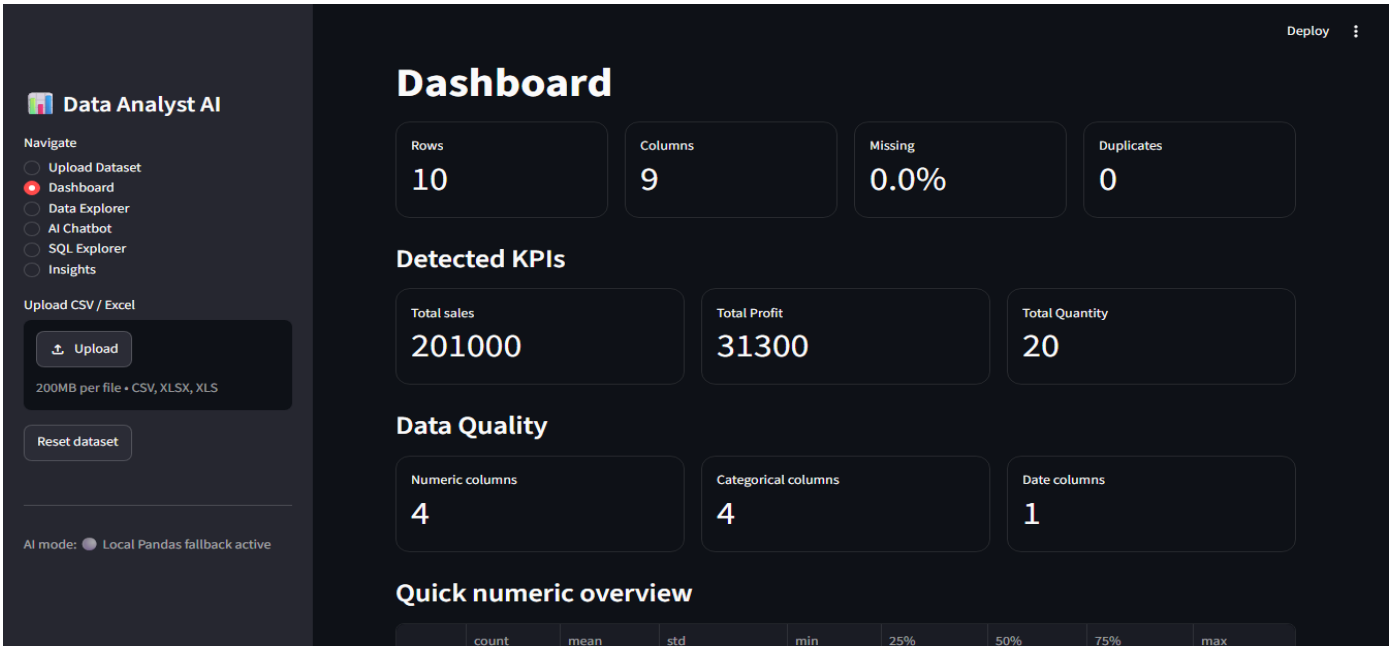
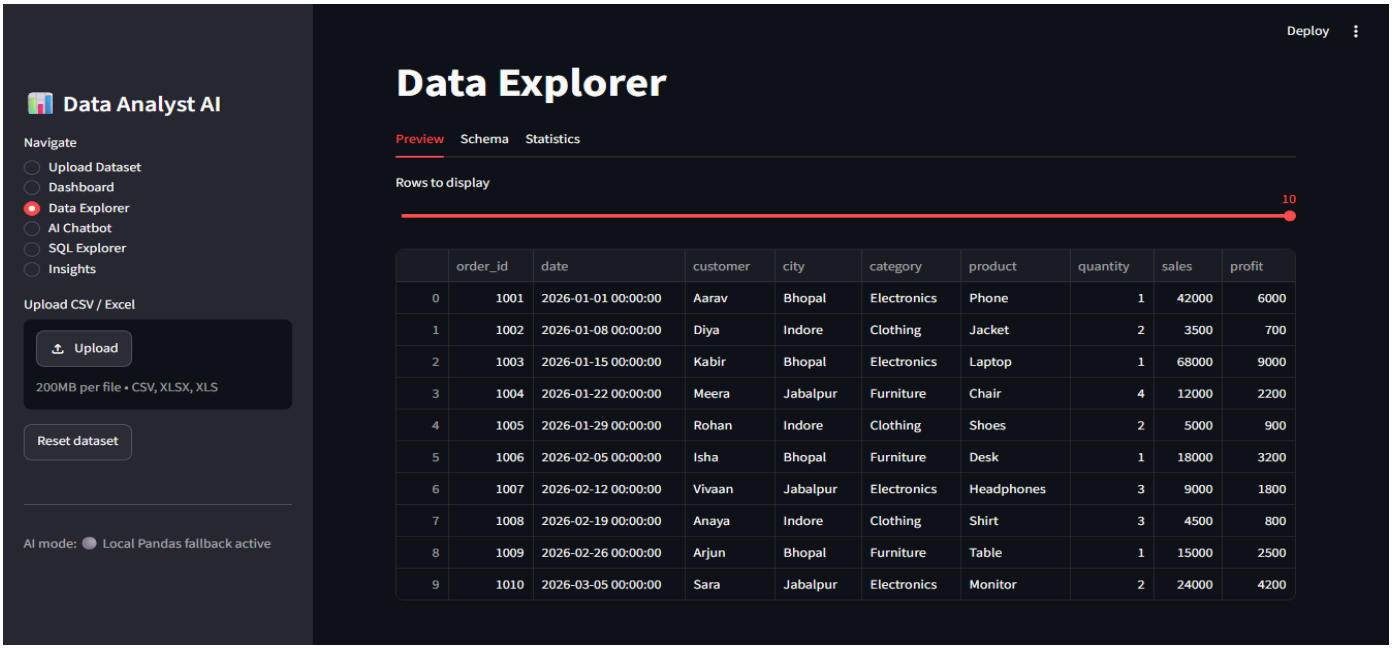


Figure 4. Executive KPI Dashboard: Total sales, profit, quantity metrics, data quality indicators, and numeric summary.



	order_id	date	customer	city	category	product	quantity	sales	profit
0	1001	2026-01-01 00:00:00	Aarav	Bhopal	Electronics	Phone	1	42000	6000
1	1002	2026-01-08 00:00:00	Diya	Indore	Clothing	Jacket	2	3500	700
2	1003	2026-01-15 00:00:00	Kabir	Bhopal	Electronics	Laptop	1	68000	9000
3	1004	2026-01-22 00:00:00	Meera	Jabalpur	Furniture	Chair	4	12000	2200
4	1005	2026-01-29 00:00:00	Rohan	Indore	Clothing	Shoes	2	5000	900
5	1006	2026-02-05 00:00:00	Isha	Bhopal	Furniture	Desk	1	18000	3200
6	1007	2026-02-12 00:00:00	Vivaan	Jabalpur	Electronics	Headphones	3	9000	1800
7	1008	2026-02-19 00:00:00	Anaya	Indore	Clothing	Shirt	3	4500	800
8	1009	2026-02-26 00:00:00	Arjun	Bhopal	Furniture	Table	1	15000	2500
9	1010	2026-03-05 00:00:00	Sara	Jabalpur	Electronics	Monitor	2	24000	4200

Figure 5. Interactive Data Explorer: Multi-tab view showing preview of loaded records, schema datatypes, and missing value analysis.



Figure 6. Conversational AI Chatbot: Natural language query translated to safe SQL with narrative summary and interactive Plotly chart.

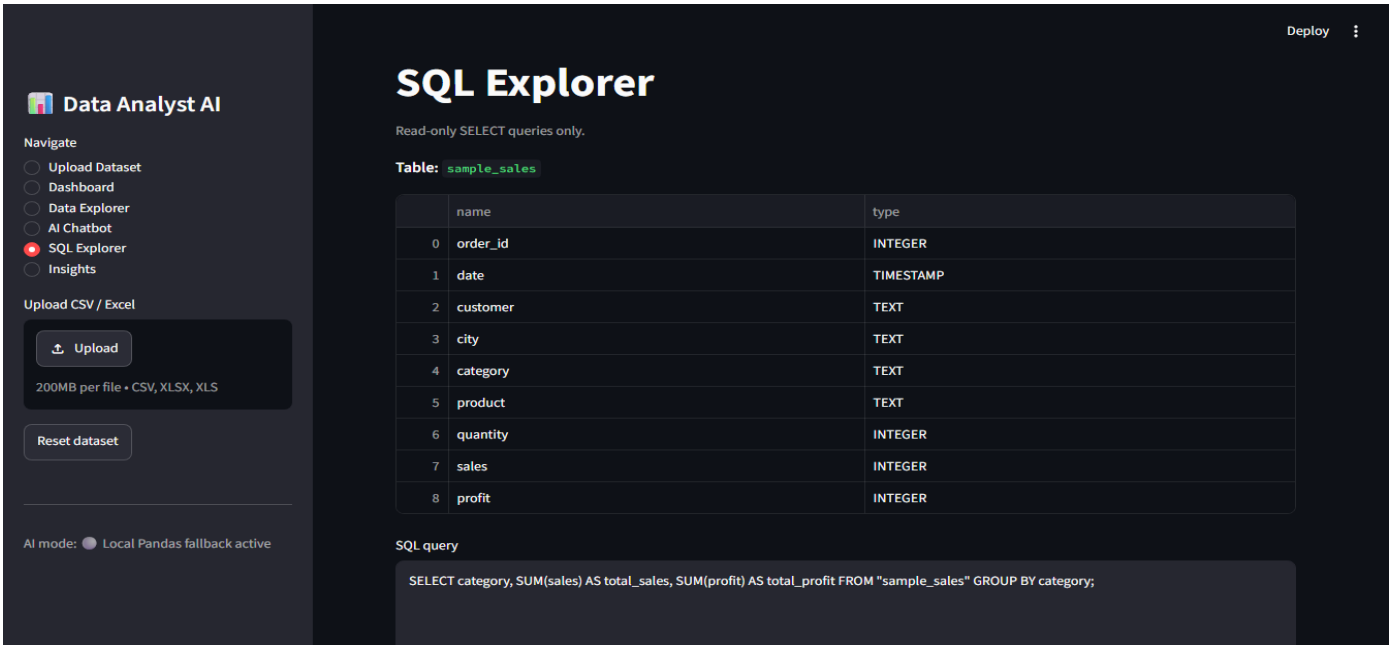


Figure 7. SQL Explorer Sandbox: Read-only SQL query validation, tabular result display, and automatic chart generation.

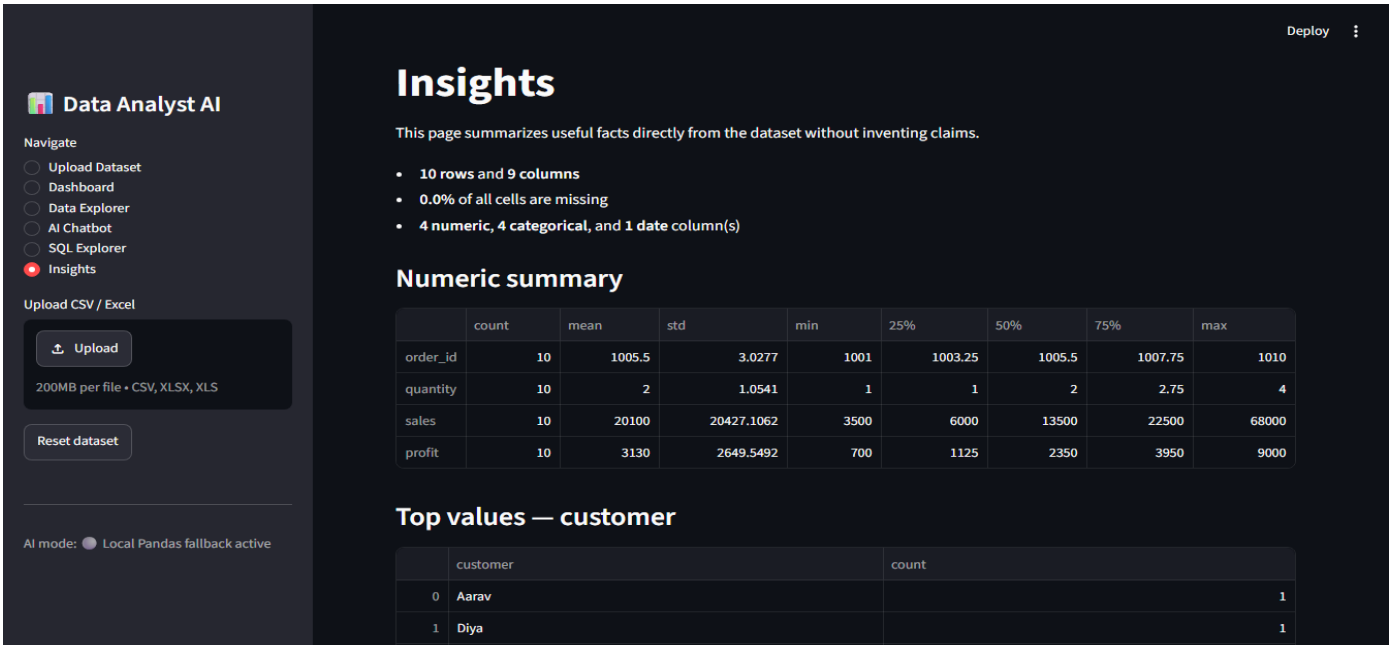


Figure 8. Automated Insights Page: Statistical overview of numerical columns, frequency distribution of categories, and top-value charts.

7. Challenges Faced and How Resolved

Challenge	Resolution
LLMs hallucinating invalid or destructive SQL syntax.	Enforced an AST-based parser using sqlparse to inspect the query structure, reject multiple statements, and block all mutation keywords (DROP, DELETE, UPDATE).
Third-party LLM rate limits and token exhaustion causing app paralysis.	Engineered a local deterministic Pandas fallback engine that intercepts core statistical questions (sum, average, count, min, max, group by) and computes answers directly.
Tabular header anomalies and inconsistent data types across user files.	Constructed an automated data cleaner that strips whitespaces, converts spaces to underscores, infers date/numeric types, and reports changes in an audit log.
Querying SQLite without persisting unauthorized data across sessions.	Implemented dynamic session-bound table generation with sanitized naming conventions, ensuring complete data isolation between user files.
Interpreting query results for non-technical decision makers.	Coupled query execution with an LLM narrative generator and dynamic Plotly visualization mapper that detects continuous vs. discrete data.

7.1 Known limitations

- Natural-language-to-SQL accuracy depends on column name clarity; ambiguous headers may require manual user clarification.
- The current local fallback engine covers single-table aggregations; nested subqueries require active LLM mode.
- Very large datasets (> 1 million records) may encounter in-memory Pandas constraints on standard consumer hardware.

7.2 Future improvements

- **Multi-Table Relational Joins:** Support simultaneous upload of multiple tables with automated foreign key linking.
- **Local Open-Source LLMs:** Integrate Ollama (Llama 3 / Mistral) for 100% offline, privacy-guaranteed on-premise execution.
- **Exportable Executive Reports:** One-click generation of PDF / Word analytical summary reports with embedded visualizations.
- **Machine Learning Insights:** Incorporate automated anomaly detection and time-series forecasting modules.

8. References

- OpenAI. (2024). *GPT-4o and GPT-4o-mini System Architecture and Function Calling Documentation*. https://platform.openai.com/docs
- Streamlit. (2024). *Streamlit Documentation: Building Interactive Python Data Applications*. https://docs.streamlit.io
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
- SQLite Development Team. (2024). *SQLite In-Memory and Embedded Relational Database Architecture*. https://www.sqlite.org
- Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library*. https://plotly.com/python
- sqlparse Developers. (2024). *Non-validating SQL parser module for Python*. https://github.com/andialbrecht/sqlparse

9. Ethical Compliance Declaration

I confirm that:

[ ] All dataset usage complies with the source licence.

[ ] No personally identifiable information (PII) is stored without consent.

[ ] AI tools used for code/writing are disclosed in the Appendix.

[ ] This project complies with the DPDP Act 2023 where applicable.

Learner Signature: _____ **Date:** _____

Appendix: AI Tools Disclosure

OpenAI API (GPT-4o-mini) is utilized within the application to perform schema-grounded natural-language-to-SQL translation and narrative result summarization. An AI assistant was employed to help draft and format this synopsis from the project's codebase, live application outputs, and the academic synopsis template.

GitHub Repository

Source code: **https://github.com/atishkr25/data-analyst-ai-chatbot**